

Quantum Fourier Transform: Theory and Implementation

Morten Hjorth-Jensen

May 2021

Contents

1	Introduction and Background	5
1.1	Classical Fourier Transform	5
1.2	Quantum States and Qubits	5
1.3	Quantum Fourier Transform	5
1.4	Matrix Form	6
2	Circuit Implementation of QFT	7
2.1	QFT Circuit Structure	7
2.2	Quantum Gates Used	7
2.3	QFT on 3 Qubits	7
3	Mathematical Foundations and Motivation	9
3.1	Introduction	9
3.2	Classical Discrete Fourier Transform	9
3.3	Quantum Fourier Transform	9
3.4	Comparison with Classical DFT	10
3.5	Unitary Nature of QFT	10
3.6	PennyLane Setup and Basic Example	10
4	Quantum Circuit Implementation	11
4.1	High-Level Structure	11
4.2	QFT on 3 Qubits	11
4.3	Controlled Phase Rotation	11
4.4	Circuit Diagram	12
4.5	Python Code with PennyLane	12
4.6	Complexity	12
4.7	Summary	12
5	Inverse QFT and Phase Estimation	15
5.1	The Inverse QFT	15
5.2	Quantum Phase Estimation (QPE)	15
5.3	Circuit Diagram (3-bit QPE)	16
5.4	PennyLane Implementation of QPE	16
5.5	Accuracy and Readout	17
5.6	Summary	17
5.7	Amplitude Estimation	17
5.8	PennyLane Example: Energy Estimation	17
5.9	Summary	18
5.10	Example: Approximate QFT in PennyLane	18

5.11 Resource Summary	18
5.12 Summary	19
5.13 Code 3: Quantum Phase Estimation (QPE)	19
5.14 Code 4: Approximate QFT Implementation	19
5.15 Execution Tips	20
5.16 Further Reading	20
5.17 Summary	20

Chapter 1

Introduction and Background

The Quantum Fourier Transform (QFT) is a central building block in quantum algorithms such as Shor's factoring algorithm and quantum phase estimation. It is the quantum analogue of the discrete Fourier transform (DFT) and enables efficient extraction of periodicity from quantum states.

1.1 Classical Fourier Transform

The classical Discrete Fourier Transform (DFT) on a vector $x = (x_0, \dots, x_{N-1})$ is given by:

$$\tilde{x}_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{2\pi i j k / N}$$

It is a unitary transformation and can be implemented in $O(N \log N)$ via the Fast Fourier Transform.

1.2 Quantum States and Qubits

A quantum register of n qubits encodes a vector in 2^n -dimensional Hilbert space:

$$|\psi\rangle = \sum_{j=0}^{2^n-1} x_j |j\rangle$$

where x_j are complex amplitudes satisfying $\sum |x_j|^2 = 1$.

1.3 Quantum Fourier Transform

The QFT on an n -qubit state transforms:

$$|j\rangle \rightarrow \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i j k / 2^n} |k\rangle$$

This unitary matrix has exponential size, but can be implemented in $O(n^2)$ quantum gates.

1.4 Matrix Form

For $n = 2$ qubits, the QFT matrix is:

$$\frac{1}{2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{bmatrix}$$

Chapter 2

Circuit Implementation of QFT

2.1 QFT Circuit Structure

The QFT is decomposed into Hadamard and controlled phase shift gates. For n qubits, the standard structure is:

1. Apply Hadamard to the most significant qubit.
2. Apply controlled- R_k gates to encode phase shifts.
3. Repeat for remaining qubits.
4. Apply SWAP gates to reverse the order.

2.2 Quantum Gates Used

- **Hadamard gate:** $H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$
- **Phase shift gate:** $R_k = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{bmatrix}$

2.3 QFT on 3 Qubits

Example for $n = 3$ qubits:

1. Apply H to qubit 0.
2. Apply R_2 (ctrl:1, tgt:0), R_3 (ctrl:2, tgt:0).
3. Repeat with Hadamard on qubit 1, R_2 from qubit 2.
4. Hadamard on qubit 2.
5. Apply SWAPs: swap(0,2).

Chapter 3

Mathematical Foundations and Motivation

3.1 Introduction

The Quantum Fourier Transform (QFT) is a key component in many quantum algorithms, notably Shor's factoring algorithm and quantum phase estimation. It serves as the quantum analogue of the classical Discrete Fourier Transform (DFT), but operates exponentially faster due to the principles of quantum parallelism and entanglement.

3.2 Classical Discrete Fourier Transform

Given a vector of N complex numbers $\{x_0, x_1, \dots, x_{N-1}\}$, the DFT is defined as:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-2\pi i k n / N}, \quad k = 0, 1, \dots, N-1$$

The inverse transform is:

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k e^{2\pi i k n / N}$$

This can be expressed in matrix form:

$$\vec{X} = F_N \vec{x}, \quad \text{where} \quad (F_N)_{jk} = \frac{1}{\sqrt{N}} e^{2\pi i j k / N}$$

3.3 Quantum Fourier Transform

Let $N = 2^n$. The QFT acts on the state:

$$|x\rangle = |x_1 x_2 \dots x_n\rangle$$

and transforms it into:

$$\text{QFT } |x\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i x k / N} |k\rangle$$

The QFT is unitary and can be decomposed into a series of Hadamard and controlled phase gates, allowing for an efficient $O(n^2)$ implementation.

3.4 Comparison with Classical DFT

While the classical FFT runs in $O(N \log N)$ time, the QFT runs in $O(n^2) = O(\log^2 N)$ gate operations—an exponential improvement, crucial in quantum speedups for specific problems.

3.5 Unitary Nature of QFT

The QFT matrix F_N is unitary:

$$F_N^\dagger F_N = I$$

This guarantees that QFT is a valid quantum operation (i.e., norm-preserving and reversible).

3.6 PennyLane Setup and Basic Example

To experiment with QFT, we can use the PennyLane Python library.

Installation

```
pip install pennylane
```

Classical DFT vs Quantum Fourier Transform

```
“python import pennylane as qml from pennylane import numpy as np
    n_qubits = 3 dev = qml.device("default.qubit", wires = n_qubits)
    def apply_qft(wires) : qml.templates.QFT(wires = wires)
    @qml.qnode(dev) def qft_circuit() : qml.BasisState(np.array([1, 0, 1]), wires = range(n_qubits))appl
range(n_qubits))return qml.state()
    state = qft_circuit()print("QFT output state : ", state)
```

Chapter 4

Quantum Circuit Implementation

4.1 High-Level Structure

The Quantum Fourier Transform can be efficiently implemented using a combination of Hadamard gates and controlled phase gates. Given an n -qubit state $|x\rangle$, the QFT circuit transforms it into the state:

$$\frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i x k / 2^n} |k\rangle$$

This is achieved through a decomposition into a quantum circuit of $O(n^2)$ gates.

4.2 QFT on 3 Qubits

We illustrate the circuit for $n = 3$. The transformation proceeds as follows:

1. Apply a Hadamard gate to the first qubit.
2. Apply controlled- R_k gates (phase gates) for $k = 2, 3, \dots$ to encode phase interference.
3. Repeat recursively on the remaining qubits.
4. Apply a qubit swap at the end to reverse the order of qubits.

4.3 Controlled Phase Rotation

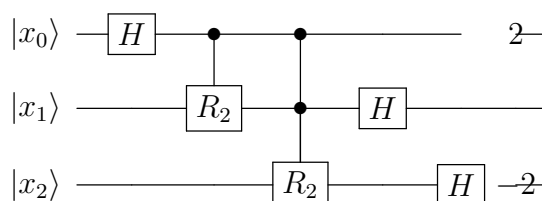
The controlled phase rotation gate R_k is defined as:

$$R_k = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i / 2^k} \end{bmatrix}$$

This gate introduces the correct phase into the wavefunction based on the binary digits of the input number.

4.4 Circuit Diagram

The QFT circuit for 3 qubits is shown below (read top to bottom):



4.5 Python Code with PennyLane

We now implement a full QFT circuit manually for 3 qubits using PennyLane.

```
import pennylane as qml
from pennylane import numpy as np
import math

dev = qml.device("default.qubit", wires=3)

def qft_rotations(wires):
    if len(wires) == 0:
        return
    head, *tail = wires
    qml.Hadamard(wires=head)
    for i, qubit in enumerate(tail):
        qml.ctrl(qml.PhaseShift, control=qubit)(math.pi / 2**(i+1),
        ↪ wires=head)
    qft_rotations(tail)

def swap_registers(wires):
    for i in range(len(wires) // 2):
        qml.SWAP(wires=[wires[i], wires[-i - 1]])

@qml.qnode(dev)
def qft_custom():
    qml.BasisState(np.array([1, 0, 1]), wires=range(3))
    qft_rotations(wires=[0, 1, 2])
    swap_registers(wires=[0, 1, 2])
    return qml.state()

print("Custom QFT state:", qft_custom())
```

4.6 Complexity

- The QFT requires $O(n^2)$ gates. - Approximate QFT implementations can reduce complexity by omitting small-angle phase gates.

4.7 Summary

The circuit implementation of QFT is surprisingly efficient, enabling its use in several key quantum algorithms. Understanding the gate-level construction deepens our under-

standing of quantum information flow during Fourier transformation.

Chapter 5

Inverse QFT and Phase Estimation

5.1 The Inverse QFT

The inverse Quantum Fourier Transform (IQFT) is simply the Hermitian adjoint of the QFT. Since the QFT is unitary, its inverse is:

$$\text{QFT}^\dagger = \text{QFT}^{-1}$$

Circuit Construction

To reverse the QFT circuit:

1. Swap the order of gates.
2. Replace Hadamard gates with Hadamard gates (self-inverse).
3. Replace each R_k gate with its conjugate transpose $R_k^\dagger$:

$$R_k^\dagger = \begin{bmatrix} 1 & 0 \\ 0 & e^{-2\pi i/2^k} \end{bmatrix}$$

5.2 Quantum Phase Estimation (QPE)

Quantum Phase Estimation is a core quantum algorithm that uses the QFT to estimate the phase ϕ in:

$$U |u\rangle = e^{2\pi i\phi} |u\rangle$$

Algorithm Overview

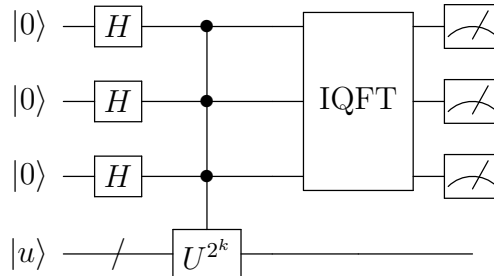
Given an eigenvector $|u\rangle$ of a unitary U , the QPE algorithm estimates the phase ϕ using:

1. Apply Hadamards to an n -qubit register.
2. Use controlled- U^{2^k} gates to encode phase.
3. Apply inverse QFT to extract the binary digits of ϕ .

5.3 Circuit Diagram (3-bit QPE)

The general circuit has:

- Top register: n qubits to encode phase.
- Bottom register: one eigenstate $|u\rangle$.



5.4 PennyLane Implementation of QPE

```
import pennylane as qml
from pennylane import numpy as np

n_estimation = 3
dev = qml.device("default.qubit", wires=n_estimation + 1)

def apply_controlled_unitaries(U, wires):
    for i, ctrl in enumerate(wires[:-1]):
        power = 2**i
        for _ in range(power):
            qml.ctrl(U, control=ctrl)(wires[-1])

def inverse_qft(wires):
    swap_registers(wires)
    for i in reversed(range(len(wires))):
        for j in range(i):
            qml.ctrl(qml.PhaseShift, control=wires[j])(-np.pi / 2**(i -
                ↪ j), wires=wires[i])
        qml.Hadamard(wires=wires[i])

@qml.qnode(dev)
def qpe():
    target_wire = n_estimation
    for i in range(n_estimation):
        qml.Hadamard(wires=i)

    qml.RZ(2 * np.pi * 0.375, wires=target_wire) # Assume phase = 0.375

    apply_controlled_unitaries(qml.RZ, list(range(n_estimation)) + [
        ↪ target_wire])
    inverse_qft(list(range(n_estimation)))
    return qml.probs(wires=range(n_estimation))

probs = qpe()
print("Measured phase state:", probs)
```

5.5 Accuracy and Readout

The accuracy of phase estimation improves with the number of qubits in the control register. For n qubits, we obtain the n most significant bits of ϕ .

5.6 Summary

The inverse QFT is essential for extracting phase information from quantum states. Combined with controlled unitary evolution, it enables phase estimation—a foundational subroutine for factoring, quantum simulations, and eigenvalue problems.

ics, eigenvalues of unitary evolutions encode properties like energy levels. QPE, powered by QFT, is used to extract these values.

Hamiltonian Simulation

Let H be a Hamiltonian. The evolution $U = e^{-iHt}$ has eigenvectors $|u\rangle$ and eigenvalues e^{-iEt} . QPE yields estimates of E .

Applications

- Electronic structure problems - Dynamics of molecules - Energy spectrum estimation

5.7 Amplitude Estimation

Quantum Amplitude Estimation (QAE) uses QFT for estimating probabilities (amplitudes) more efficiently than classical sampling.

Basic Setup

Prepare a state:

$$|\psi\rangle = \sqrt{a}|1\rangle + \sqrt{1-a}|0\rangle$$

QAE estimates a using Grover-like iterations and QFT.

5.8 PennyLane Example: Energy Estimation

We simulate a simple diagonal Hamiltonian using QPE.

```
import pennylane as qml
from pennylane import numpy as np

n_qubits = 2
dev = qml.device('default.qubit', wires=n_qubits + 1)

def controlled_unitary(t):
    def unitary():
        qml.PhaseShift(2 * np.pi * 0.3 * t, wires=n_qubits)
    return unitary

@qml.qnode(dev)
```

```

def energy_estimation():
    # Apply Hadamards to estimation qubits
    for i in range(n_qubits):
        qml.Hadamard(wires=i)
    # Prepare eigenstate for target qubit
    qml.PauliX(wires=n_qubits)
    # Apply controlled- $U^{2^j}$ 
    for i in range(n_qubits):
        unitary = controlled_unitary(2**i)
        qml.ctrl(unitary, control=i)()
    # Inverse QFT
    inverse_qft(range(n_qubits))
    return qml.probs(wires=range(n_qubits))

probs = energy_estimation()
print('Measured energy phase:', probs)

```

5.9 Summary

The QFT enables exponential speedups in various quantum algorithms by exposing hidden periodicity, encoding eigenvalues, and amplifying amplitudes. Its applications span number theory, chemistry, and quantum finance, highlighting its centrality to quantum computing.

large k are highly sensitive to noise. Removing them improves fidelity on NISQ devices.

Noise-Aware Compilation

Use device-specific compilers (e.g., Qiskit, tket) that minimize gate errors and optimize qubit layout.

5.10 Example: Approximate QFT in PennyLane

We modify our QFT to drop R_k gates with angle $< \epsilon$:

```

def approximate_qft(wires, epsilon=1e-2):
    if len(wires) == 0:
        return
    head, *tail = wires
    qml.Hadamard(wires=head)
    for i, qubit in enumerate(tail):
        angle = np.pi / 2**(i+1)
        if angle >= epsilon:
            qml.ctrl(qml.PhaseShift, control=qubit)(angle, wires=head)
    approximate_qft(tail)

```

This dramatically reduces depth for large n , making QFT feasible for hardware implementation.

5.11 Resource Summary

- **Exact QFT:** $O(n^2)$ gates, accurate, but deep circuit.

- **Approximate QFT:** $O(n \log(1/\epsilon))$ gates, fast and NISQ-friendly.
- **Inverse QFT:** Similar structure, used in readout stages.

5.12 Summary

Optimizing QFT for real-world execution is essential for practical quantum computing. Approximate QFTs maintain algorithmic utility while reducing quantum hardware demands. These optimizations are vital for algorithms targeting NISQ systems or requiring high repetition rates.

```
sed(range(len(wires))): for j in range(i): qml.ctrl(qml.PhaseShift, control=wires[j])(-
np.pi / 2**(i - j), wires=wires[i]) qml.Hadamard(wires=wires[i])
```

5.13 Code 3: Quantum Phase Estimation (QPE)

```
n_estimation = 3
dev = qml.device('default.qubit', wires=n_estimation + 1)

def apply_controlled_unitaries(U, wires):
    for i, ctrl in enumerate(wires[:-1]):
        power = 2**i
        for _ in range(power):
            qml.ctrl(U, control=ctrl)(wires[-1])

@qml.qnode(dev)
def qpe():
    target_wire = n_estimation
    for i in range(n_estimation):
        qml.Hadamard(wires=i)
        qml.RZ(2 * np.pi * 0.375, wires=target_wire)
        apply_controlled_unitaries(qml.RZ, list(range(n_estimation)) + [
            ↪ target_wire])
    inverse_qft(list(range(n_estimation)))
    return qml.probs(wires=range(n_estimation))

print('Measured phase state:', qpe())
```

5.14 Code 4: Approximate QFT Implementation

```
def approximate_qft(wires, epsilon=1e-2):
    if len(wires) == 0:
        return
    head, *tail = wires
    qml.Hadamard(wires=head)
    for i, qubit in enumerate(tail):
        angle = np.pi / 2**(i+1)
        if angle >= epsilon:
            qml.ctrl(qml.PhaseShift, control=qubit)(angle, wires=head)
    approximate_qft(tail)
```

5.15 Execution Tips

- All examples can run on `default.qubit`, the standard PennyLane simulator.
- Use `qml.draw()` to visualize circuits.
- To run on real hardware (e.g., IBMQ, Xanadu X8), swap the device line with the appropriate plugin and set API keys.
- Use `qml.state()` or `qml.probs()` to inspect output.
- PennyLane supports backpropagation, enabling QFT-based circuits to be optimized or trained.

5.16 Further Reading

- PennyLane Documentation: <https://docs.pennylane.ai>
- Nielsen and Chuang, *Quantum Computation and Quantum Information*
- Kitaev's Phase Estimation Algorithm (original source)
- Qiskit textbook: <https://qiskit.org/textbook>

5.17 Summary

This appendix compiles all code needed to simulate and visualize QFT and its applications using PennyLane. With these examples, readers can replicate core quantum algorithms on simulators or real hardware with minimal changes.